Suporte e Infraestrutura

Repositório Central de Concursos de TIC · UFRRJ & Transpetro 2026

Caderno Consolidado de Suporte e Infraestrutura — teoria completa, mnemônicos e pegadinhas de prova.

Gerado automaticamente em 09/11/2026.

Sumário

- Infraestrutura — T02: Arquitetura e Hardware de Servidores
- Infraestrutura T03 — Armazenamento de Dados (SAN, NAS, DAS, RAID)
- Infraestrutura T04 — Virtualização, Cluster e Nuvem
- Infraestrutura T05 — Servidores de Aplicação (IIS e Apache)
- Infraestrutura — T06: Administração de Sistemas Operacionais (Linux e Windows)
- Infraestrutura T07 — Armazenamento Orientado a Objetos (S3)

Infraestrutura — T02: Arquitetura e Hardware de Servidores

■ Visão Geral

A arquitetura de hardware em servidores corporativos exige alta disponibilidade, taxa de transferência massiva e controle de integridade. As bancas cobram a divisão funcional dos barramentos do sistema, as gerações e características das memórias RAM (DDR3, DDR4 e DDR5) e as interfaces de barramento de alto desempenho.

Estrutura do Barramento do Sistema (System Bus)

O barramento do sistema é a via de comunicação física que interconecta a Unidade Central de Processamento (CPU), a memória principal e os dispositivos de entrada e saída (E/S). Ele é dividido funcionalmente em três barramentos especializados:

Barramento	O que transporta?	Direção do Fluxo	Função Crítica em Prova
	Barramento de Dados (*Data Bus*)	Os dados e instruções propriamente ditos	**Bidirecional**
	Barramento de Endereços (*Address Bus*)	O endereço da posição de memória ou de E/S	**Unidirecional** (da CPU para a memória/periférico)
	Barramento de Controle (*Control Bus*)	Sinais de sincronismo, temporização e comando	**Bidirecional** (misto)

■■ O Barramento de Endereços NÃO transporta dados!

A banca adora afirmar que o barramento de endereços "transfere os dados contidos na memória". Isso é falso! Ele transporta exclusivamente o **número da casa** (o endereço de 32 ou 64 bits). Quem viaja dentro da casa (o dado) é transportado exclusivamente pelo **Barramento de Dados**.

Memórias RAM: Evolução e Padrões DDR (Double Data Rate)

A memória RAM (Random Access Memory) é o armazenamento primário e volátil do sistema. A tecnologia padrão de mercado é a **SDRAM DDR** (*Double Data Rate Synchronous Dynamic RAM*), cuja característica mestra é transferir dados **duas vezes por ciclo de clock** (na subida e na descida do pulso).

2.1 Comparativo das Gerações DDR

Padrão	Tensão Operacional (Volts)	Eficiência Energética	Faixa Típica de Taxa de Transferência	Recurso Chave de Servidor
	DDR3	1,5V (ou 1,35V na DDR3L)	Alta dissipação de calor	800 a 2.133 MT/s
	DDR4	1,2V	20% mais econômica que DDR3	1.600 a 3.200 MT/s
	DDR5	1,1V	Consumo mínimo com PMIC no módulo	4.800 a 8.400+ MT/s

■ Incompatibilidade Física e Elétrica

Módulos DDR de gerações diferentes **não são retrocompatíveis**. O entalhe físico (*notch*) no conector dos pinos muda de posição a cada geração para impedir fisicamente a inserção de um módulo DDR4 em um slot DDR3 ou DDR5. Além disso, as tensões são incompatíveis (tentar ligar causaria dano elétrico).

Confiabilidade Corporativa: Memórias com ECC e Registradas (RDIMM)

Em servidores e storages críticos, o travamento por corrupção de bits (*bit flip*) causado por radiação cósmica ou variações elétricas é inaceitável. Por isso, utilizam-se tecnologias específicas:

- **Memórias ECC (*Error-Correcting Code*):** Possuem bits adicionais por palavra de memória para detectar e **corrigir automaticamente erros de bit único (*Single-Bit Error*)** e detectar erros de bit duplo (*Double-Bit Error*).
- **Memórias Registradas / Buffering (RDIMM / LRDIMM):** Possuem um registrador de hardware entre os chips de DRAM e o controlador de memória, reduzindo a carga elétrica no barramento e permitindo a instalação de dezenas de módulos de alta densidade sem instabilidade.

■ Mnemônico dos Barramentos

D-E-C:

Dados: O que é levado (Bidirecional).

Endereço: Aonde é levado (Unidirecional, da CPU).

Controle: Como e Quando é levado (Temporização e comando).

Infraestrutura T03 — Armazenamento de Dados (SAN, NAS, DAS, RAID)

■■ **INFO**

Armazenamento de Dados Corporativo: arquitetura de redes de armazenamento (DAS, NAS e SAN), protocolos Fibre Channel, iSCSI e FCoE, matrizes redundantes de discos (RAID 0, 1, 5, 6 e 10), sistemas de arquivos, replicação e técnicas de proteção de dados.

Do que se trata?

Existem três arquiteturas de armazenamento que a FGV/CESPE alternam o tempo todo — e o ponto que decide a questão é **o nível em que o dado é acessado**: **bloco**, **arquivo** ou **direto do disco**.

Arquitetura	Nível de acesso	Como conecta	Protocolos típicos	Custo/complexidade
	DAS (Direct Attached Storage)	disco (bloco)	liga direto em um servidor ("gaveta" de disco)	IDE, SCSI, SATA, SAS, FC ponto a ponto
	SAN (Storage Area Network)	**bloco**	rede dedicada de alta velocidade	**Fibre Channel**, iSCSI, FCoE
	NAS (Network Attached Storage)	**arquivo**	rede corporativa (Ethernet/TCP/IP)	**CIFS/SMB** (Windows), **NFS** (Linux)

■ **RESUMO**

SAN pensa em "blocos", NAS pensa em "arquivos", DAS não pensa — pendura direto no servidor. Em prova: quem diz que SAN serve arquivos ou tem sistema operacional de servidor de arquivos está descrevendo o NAS — e a assertiva é falsa.

O protocolo: onde a banca brinca

- **Fibre Channel (FC)**: protocolo de SAN que usa **fibra óptica OU cobre** — "fibra" no nome não significa "só fibra".
- **iSCSI**: mapeia comandos **SCSI sobre TCP/IP** — a SAN virando "IP SAN". Mais econômico que FC por reaproveitar switches IP/Ethernet, mas introduz maior latência/overhead (não é o padrão para bases de dados ultra críticas de alto IOPS).
- **FCoE** (Fibre Channel over Ethernet): leva tráfego FC **e** IP juntos sobre **Ethernet** (convergência — um só enlace para storage + rede).
- **FCIP** (FC over IP): encapsula tramas FC em pacotes IP para **estender** a SAN a longas distâncias (replicação/continuidade).
- **SAN por Ethernet**: a infra pode ser **Gigabit Ethernet** — não precisa ser FC. Mas na **maioria dos modelos atuais** o padrão é FC com fibra.

■■ PEGADINHA

O par clássico da banca: **IP SAN usa FCIP e iSCSI**. Alternativas tentam vender FCoE/FC/outras misturas como se fossem "os protocolos primários de IP SAN". E mais: "Fibre Channel over Ethernet" (FCoE) e "FC over IP" (FCIP) são **coisas diferentes** — convergir numa rede e estender sobre IP não é a mesma operação.

A Arquitetura em Camadas do Fibre Channel (FC-0 a FC-4)

O modelo Fibre Channel define 5 camadas funcionais (numeradas de baixo para cima, de FC-0 a FC-4):

Camada	Nome	Função Principal	Padrões e Tecnologias
	FC-4	*Upper Layer Protocol (ULP)*	Interface com os aplicativos de nível superior
	FC-3	*Common Services*	Serviços comuns compartilhados entre múltiplas portas
	FC-2	*Framing / Signaling Protocol*	Coração do transporte: segmentação de blocos em quadros (frames), controle de fluxo e roteamento
	FC-1	*Transmission Protocol*	Codificação/decodificação e sincronização de sinal
	FC-0	*Physical Interface*	Camada física: meios de transmissão, conectores e sinalização

SAN (*Storage Area Network*): o padrão-ouro corporativo

- Rede **dedicada e de alta velocidade** para conectar servidores a dispositivos de armazenamento (**nível de bloco**).
- **Independente do sistema operacional** — servidores variados acessam o mesmo storage.
- Pode ser **compartilhada entre servidores** ou dedicada a um único (local/remoto).
- **Custo e complexidade mais altos** que NAS (infra estruturada, switches FC dedicados, protocolo especializado) — por isso vai para **sistemas críticos**.
- A malha SAN (*SAN Fabric*) = switches FC + roteadores/gateways + cabos interconectando tudo.
- **Não tem "sistema operacional próprio"** nem é servidor de arquivos — isso é do NAS.

■ ALERTA

SAN **não é menos complexa** que NAS e **não usa comutadores ethernet como regra** quando o padrão é FC. Também **não é "só fibra óptica"**: FC roda sobre fibra **ou** cobre, e há SAN Ethernet (iSCSI). Pegadinha típica: "NAS é mais caro que SAN" → **falso** (é o contrário).

Segurança em SAN — os três mecanismos: a SAN é uma rede rica em dados críticos, então ela precisa de controle de acesso próprio, feito por três camadas que a banca confunde o tempo todo:

Mecanismo	O que faz	Analogia

	LUN (Logical Unit Number)	dá um **número de identificação** a um pedaço lógico do storage
	Masking	controla **quais servidores** podem enxergar/acessar aquela LUN
	Zoning	divide a rede SAN em **zonas**: só enxerga quem está na mesma zona

■ DICA

Sempre que a questão falar em "evitar acesso não autorizado em SAN", junte os três: **LUN identifica, masking libera servidor, zoning segmenta a rede**. Zoneamento **não** existe em DAS — e "SAN precisa de zoneamento" é correta, "DAS precisa de zoneamento" é falsa.

Fabric Zoning na Prática: O Cenário Clássico de Prova

Em ambientes corporativos com múltiplos ambientes (Produção, Desenvolvimento) e recursos de proteção compartilhados, o zoneamento (*Fabric Zoning*) opera com segmentação estrita:

- **Storage P** (Produção) $\rightarrow$ isolado na **Zona A**.
- **Storage D** (Desenvolvimento) $\rightarrow$ isolado na **Zona B**.
- **Tape Library** (Unidade de Fita para backup) $\rightarrow$ isolada na **Zona C**.

■■ PEGADINHA

Em quais zonas os servidores de Produção precisam ser inseridos para realizar backup?

Para ler os dados de negócio, os servidores de produção precisam acessar o *Storage P* (**Zona A**). Para enviar as cópias para a fita, precisam acessar a *Tape Library* (**Zona C**). Portanto, os servidores de produção **devem pertencer obrigatoriamente a ambas as zonas: Zona A e Zona C**. Um host pode e deve participar de múltiplos zoneamentos para desempenhar papéis combinados!

NAS: o "compartilhar pastas" da empresa (19 questões)

- **File level**: entrega **arquivos**, perfeito para "servidor de arquivos" corporativo.
- Infra quase sempre **Gigabit Ethernet + TCP/IP**.
- Protocolos: **CIFS/SMB** para ambientes Windows; **NFS** para Linux; **AFP** para ambientes Apple.
- **Sistema operacional e sistema de arquivos próprios**: quem **formata os discos e gerencia o filesystem (Ext4, Btrfs, ZFS...) é o próprio NAS**, nunca o cliente da rede — o usuário só vê pastas compartilhadas por cima.
- Custo/gestão **menores** que SAN; mais simples, porém **menor desempenho** (não disputa com SAN em IOPS).
- Ambos (NAS e SAN) podem encostar numa LAN e servir clientes heterogêneos — o que diferencia é o **nível** (arquivo × bloco), não a possibilidade de rede.

■ ALERTA

Pegadinha do filesystem: "o **cliente** formata os discos do NAS com NTFS" → **falso**; "o NAS exige **Fibre Channel** dos clientes" → **falso** (é trocar NAS por SAN). Definidor de NAS é **quem gerencia o filesystem** (o equipamento) e **em que nível expõe o acesso** (arquivo).

■■ PEGADINHA

A banca adora alternativas falsas como: "SAN é mais simples por usar SMB" → falso (SMB é NAS); "NAS tem mais desempenho por usar a rede corporativa" → falso (a rede LAN corporativa compartilha banda e tem latência); "ambas dependem de rede exclusiva" → falso; "NAS é mais escalável por ter melhor IOPS" → falso (IOPS alto é marca registrada da SAN). A correta: **SAN custa mais e atende sistemas críticos**.

■ **DICA**

Decisão rápida por expressões funcionais da banca:

Enunciado fala em *conexão direta ao host sem rede* $\rightarrow$ **DAS**.

Enunciado fala em *compartilhamento de arquivos em rede com NFS/SMB* $\rightarrow$ **NAS**.

Enunciado fala em *rede dedicada em nível de bloco de alta performance e disponibilidade* $\rightarrow$ **SAN**.

Enunciado fala em *arranjo lógico de múltiplos discos com paridade ou espelho* $\rightarrow$ **RAID**.

DAS e o teste rápido das três

- **DAS (Direct Attached Storage)**: discos ligados **direto** ao barramento ou controladora do servidor. Simples, barato, sem malha de rede compartilhada — cada servidor com sua "gaveta".
- **DAS Externo (External DAS)**: o host conecta-se diretamente a um gabinete externo de discos dedicado. Essa ligação externa pode ser feita via barramento **SCSI** ou até mesmo cabo e protocolo **Fibre Channel (FCP) ponto a ponto** (sem switches SAN intermediários). Supera os limites físicos de espaço/discos internos mantendo a simplicidade de uma conexão ponto a ponto dedicada.
- Teste do "quem atende": DAS = servidor + disco no mesmo rack/cabo dedicado; SAN = storage na **rede dedicada** (bloco); NAS = storage **na rede corporativa** entregando arquivos.

RAID: replicação, espelhamento e paridade

RAID (Redundant Array of Independent Disks) junta discos para **desempenho** e/ou **segurança**.

Nível	Técnica	Mínimo	Tolerância a falha
	RAID 0	Striping (intercala dados)	2
	RAID 1	Espelhamento (cópia idêntica)	2
	RAID 4	Striping + **paridade em disco dedicado**	3
	RAID 5	Striping + **paridade distribuída**	3
	RAID 6	Striping + **dupla paridade**	4
	RAID 10 (1+0)	Espelhamento + striping	4

■ **MEMORIZAR**

"0 só velocidade, 1 só cópia, 5 paridade espalhada, 10 espelha e risca." E os mínimos: **0/1** → **2 discos; 5** → **3; 6** → **4; 10** → **4** (dois espelhados + striping). Menos discos que o mínimo a banca marca como erro ("RAID 10 exige ao menos 3 discos" → **ERRADO**, são 4).

> **■■ PEGADINHA**
>
> Pegadinha recorrente em RAID: "RAID 5 usa paridade em disco **dedicado**" → **falso** (a paridade é **distribuída** entre todos); "RAID 6 falha com até 1 disco" → **falso** (aguenta 2). E "RAID 0 dá redundância" → nunca.

RAID 4 × RAID 5: a diferença cabe em duas palavras.

Pensa no RAID 4 como sala de repartição com um colega de quarto dedicado: os discos de dados são independentes, mas existe **um disco só seu, exclusivo, onde mora toda a paridade**. Toda gravação, antes de terminar, precisa passar por ele para atualizar a paridade — e é aí que mora o problema. Qualquer escrita nova engarrafa naquele único disco, então **gravações pequenas e independentes não conseguem rodar em paralelo**: é o famoso gargalo de gravação. A leitura, essa sim, sai em paralelo de todos os discos de dados, por isso o RAID 4 é lido bem e escrito mal. Requer **no mínimo 3 discos** — com 2 não sobra como separar "dados" e "paridade" sem virar espelhamento: paridade com dois discos daria exatamente um RAID 1.

O RAID 5 é o RAID 4 depois da reforma: mesma ideia de striping com paridade, só que os **bits de paridade são distribuídos uniformemente por todos os discos, por alternância circular**, em vez de caberem num disco só. A carga de escrita se espalha, o gargalo some e continua valendo o combo mínimo **3 discos / tolerância a 1 disco**.

> **Dois termos que a banca troca o tempo todo:** *"alternância circular / paridade distribuída"* = **RAID 5**; *"um dos discos utilizado exclusivamente para armazenar os elementos de paridade"* = **RAID 4**. Se o enunciado escolher uma dessas frases, ele já está entregando o nível — e a FGV repete essa distinção em várias provas (TJ-SE 2023).

Níveis compostos e a conta da capacidade. Com **10 discos de 1TB**, a banca adora pedir o total aproveitável — a conta é "quanto sobra depois de paridade/espelho":

RAID	10 discos de 1TB → útil	Por quê
	0	10 TB
	1	5 TB
	5	9 TB
	6	8 TB
	10	5 TB

- **RAID 50 (5+0)**: striping sobre conjuntos RAID 5 — mínimo de **6 discos** (2 conjuntos × 3).
- **RAID 60 (6+0)**: striping sobre conjuntos RAID 6 — paridade dupla em cada grupo, mais proteção e mais custo.

> **■■ PEGADINHA**
>
> **"Melhor combinação de desempenho e redundância" = RAID 10, não RAID 5**. A FGV escreve no enunciado que o RAID 10 "pode ser mais caro" justamente para você não usar custo como critério — a pergunta é desempenho **×** redundância. RAID 5 é o "melhor equilíbrio com **economia** de discos"; RAID 10 é o melhor desempenho **com** redundância. Leia o comando da questão.

> **■ MEMORIZAR**
>
> **Conta de capacidade:** RAID 0 = soma total; RAID 1 = soma ÷ 2; RAID 5 = soma – 1 disco; RAID 6 = soma – 2 discos; RAID 10 = soma ÷ 2.

Backup: as topologias que a banca monta em enunciado

Topologia = **por onde o dado do backup trafega**. São as três clássicas + a mistura:

Topologia	Caminho dos dados	Pró	Contra
	Direct-attached	storage de backup ligado **direto** no servidor protegido	simples, barato
	LAN-based	dados atravessam a **rede local** até o servidor de backup	infra já existe
	LAN-Free	dados vão pela **SAN**, sem passar pela LAN	não polui a rede local
	SAN-Based	fluxo inteiro sobre a **SAN**, dispositivo de backup compartilhado	rápido e escalável em grandes volumes

■ RESUMO

Direct-attached = cabo direto; LAN-based = backup passa pela LAN; LAN-Free/SAN-Based = backup passa pela SAN e "desafoga" a LAN. A topologia **mista** combina LAN + SAN. Em questão de falso/verdadeiro: "no direct-attached só os **metadados** vão pela LAN" e "no SAN-based o dispositivo pode ser **compartilhado** entre clientes" → certos.

■ CONEXAO

Backup com granularidade e agenda entra no **ITIL 4** (gestão de capacidade/continuidade) e na **ISO 20000** — mesma ideia de "proteger o serviço", mas com vocabulário de gestão, não de hardware.

Arquivamento vs. Backup: Estratégias Diferentes que a Prova Confunde

A FGV adora afirmar que "não há diferença entre arquivamento e backup de dados" ou que "arquivamento é apenas uma rotina de backup". **Isso é falso!**

Dimensão	Backup	Arquivamento de Dados (Archiving)
	Objetivo	Recuperação operacional rápida (Disaster Recovery / Continuidade)
	Natureza do Dado	Dados **ativos, mutáveis e operacionais** do dia a dia
	Impacto no Primário	Mantém os dados originais no storage de produção
	Tempo de Retenção	Curto a médio prazo (dias, semanas ou meses — retenção cíclica)
	Mídia Típica	Discos rápidos, replicação síncrona/assíncrona, snapshots

Virtualização de Storage (Storage Virtualization)

A **virtualização de armazenamento** cria uma camada de abstração lógica entre os servidores físicos/virtuais e os dispositivos de storage reais:

- **Pool Único Consolidado:** Permite que múltiplos subsistemas heterogêneos de SAN (de diferentes fornecedores, tecnologias e modelos de disco) sejam gerenciados e visualizados como um **único grande pool de armazenamento**.
- **Mascaramento de Complexidade:** Os servidores enxergam apenas LUNs virtuais uniformes, sem precisar conhecer a controladora física subjacente ou a marca do fabricante.
- **Data Striping vs. Espelhamento:** Em nível de controladora, a *distribuição de dados* (**data striping**) intercala blocos entre múltiplos discos com foco primordial em **desempenho de E/S**; o espelhamento (**mirroring**) duplica o dado para **tolerância a falhas**.

Proteção de dados no armazenamento: as cinco técnicas

Antes de ir para as conexões, decore o **que cada técnica faz** — a banca sempre dá o cenário e pede o nome:

Técnica	O que faz	Gatilho no enunciado
	Tokenização	troca o dado sensível por um **token** (substituto)
	Máscara dinâmica de dados	**oculta trechos na exibição** ("***"), dado original intacto e **não cifrado**
	Data shuffling	**embaralha** valores trocando-os entre registros
	Anonimização / supressão de atributos	**apaga por completo** um atributo desnecessário ou que não dá para anonimizar
	Transparent Data Encryption (TDE)	**cifra** os dados armazenados em disco

■■ PEGADINHA

Três confusões seguras de prova: (1) máscara **não** cifra — o dado segue íntegro e legível no banco; (2) tokenização **substitui** o dado todo, máscara **só esconde na leitura**; (3) supressão **remove o atributo inteiro**, ela não "embaralha nem generaliza".

■ DICA

Enunciado dizendo "dado original permanece íntegro e **não cifrado**, mas aparece parcialmente com asteriscos" → resposta é **máscara dinâmica**, nunca TDE nem tokenização.

Conexões com outras matérias

- **Redes de Computadores**: SAN/NAS herdam protocolos de rede — TCP/IP, Ethernet (T05.3), e o **Modelo OSI** (nível de bloco × arquivo ecoa camadas de apresentação/aplicação).
- **Banco de Dados**: SAN surge em prova junto com "high performance para bancos" — storage em bloco é o chão de SGBDs (Tópico 10–14 de Banco de Dados).
- **Governança**: disponibilidade e recuperação (replicação = redundância) são funções de serviço de TI que a ISO 20000/ITIL cobram.

Pegadinha final: a FGV trocou os rótulos

Cuidado com a armadilha clássica: a banca costuma misturar conceitos de **SAN e RAID** na mesma assertiva de propósito. Por exemplo, "RAID DP" (dupla paridade proprietária da NetApp) existe no mercado de storage, mas não é um nível padrão universal de implementação SAN (que se apoia nos níveis clássicos RAID 0, 1, 5, 6 e 10).

> ■ **RESUMO**
>
> **SAN = rede dedicada, nível de bloco, FC/iSCSI/FCoE/FCIP, caro e crítico. NAS = Ethernet/TCP-IP, nível de arquivo, CIFS/SMB e NFS, barato e simples. DAS = disco direto no servidor. RAID: 0 striping puro, 1 espelho, 5 paridade distribuída (3 discos), 6 dupla paridade (4), 10 espelho+strip (4).** Se a assertiva embaralhar esses pares, é falsa.

Temas de maior incidência em prova: SAN (Fibre Channel vs. iSCSI e acesso em bloco), níveis de RAID (cálculo de capacidade útil, tolerância a falhas e paridade dupla no RAID 6), diferenças entre NAS (arquivo: NFS/SMB) e SAN (bloco), e proteção de dados (mascaramento vs. criptografia TDE).

Infraestrutura T04 — Virtualização, Cluster e Nuvem

> ■■ **INFO**
>
> **Virtualização, Clusters, Contêineres e Computação em Nuvem:** hipervisores Tipo 1 (*bare-metal*) e Tipo 2 (*hosted*), consolidação de servidores, alta disponibilidade (HA) e balanceamento de carga, computação em grade (*Grid*) vs. Nuvem (NIST SP 800-145), Docker e orquestração com Kubernetes.

Do que se trata?

Pensa num prédio de escritórios. Um andar inteiro (o hardware) dividido em salas (as **máquinas virtuais**), cada uma com sua porta, seu mobiliário e seu ar-condicionado isolado. Isso é **virtualização** — abstrair recursos físicos para criar vários "computadores lógicos". Junta várias salas = **cluster**; o porteiro que decide em que sala cada visitante entra = **balanceador**. E se o prédio não é seu, é alugado por hora e aceita qualquer empresa que apareça = **nuvem**.

Virtualização: o chão de tudo

A característica que "abstrai os recursos computacionais físicos dos lógicos" é a **virtualização** — e não elasticidade, redundância, escalabilidade ou descentralização.

> ■ **RESUMO**
>
> **Elasticidade** = recursos escalados conforme a demanda. **Redundância** = duplicar componentes críticos. **Escalabilidade** = aumentar a carga sem comprometer desempenho. **Virtualização** = abstração dos recursos (base da nuvem). Se a assertiva falar de "abstração físico × lógico", é virtualização.

VMware: o vocabulário do vSphere

Recurso	Função
	vMotion
	Storage vMotion
	DRS
	HA (High Availability)
	FT (Fault Tolerance)

> **■■ PEGADINHA**
>
> A banca explora HA × FT: falou em **reinicialização automática após falha de host** → **HA**. Falou em **cópia secundária sincronizada**, sem reinício perceptível → **FT**. E o painel de monitoramento de VM no cluster HA é **falhas e respostas** — controle de admissão, datastore heartbeat, vSAN e HA proativa são os distratores.

Docker e Kubernetes: contêiner e orquestrador

Docker cria e roda contêineres individuais (a embarcação). **Kubernetes** é o capitão do navio: orquestra centenas deles — implantação automatizada, escalonamento e auto-cura. Não é hipervisor, não é sistema operacional convidado e **depende de um runtime** para executar os contêineres.

> **■ ALERTA**
>
> Pegadinha central: a FGV tenta (a) tratar Docker como orquestrador, (b) dizer que K8s elimina o runtime, (c) classificar Docker como hipervisor tipo 1. As três são **falsas**. Docker = contêinerização; orquestrador = Kubernetes.

- **Taints e tolerations**: o **nó** usa *taint* para **repelir** pods sem a combinação chave+efeito; o **pod** declara *toleration* para aceitar. A pegadinha é inverter os papéis ou marcar *node affinity*.
- **Readiness Probe**: decide quando o Pod entra no balanceamento de tráfego. **Liveness Probe** reinicia contêiner travado. HPA já criou réplicas? Ainda falta o Pod ficar "pronto".
- **HPA**: ajusta réplicas pela **utilização observada de recursos** (CPU/memória), periodicamente. Baseado em eventos = KEDA; por horário = cron-based. Não "descarta pods por diferença de configuração".
- **Pods**: a menor unidade implantável; contêineres do mesmo Pod **compartilham IP, portas, hostname (UTS) e namespaces (IPC)** — isolados de Pods de fora.
- **cgroup v2**: com o Linux cgroup v2, o kubelet usa o driver **systemd** (cgroupfs é de v1; cadvisor é monitoramento; containerd é runtime).
- **Garbage Collection**: exclusão em cascata tem **Foreground** e **Background** (V). Variáveis corretas são **MinAge, MaxPerPodContainer, MaxContainers** (não LowAge/HighPerPodContainer/HighContainers) e o kubelet usa **HighThresholdPercent e LowThresholdPercent** (não Max/MinThresholdPercent). V–F–F.
- **Rede Docker**: container em servidores diferentes → **overlay**. Bridge é no mesmo host; host/ipvlan/macvlan não conectam hosts.
- **Rancher em K3s HA**: 2 nós Linux + banco **relacional** externo (MySQL) + **balanceador de carga** + **registro DNS**.
- **Conceitos**: isolar recursos é função de **namespaces**, não de contexts (context = conjunto de acesso cluster+usuário+namespace). Kube-proxy mantém regras de rede; kubelet roda os contêineres nos Pods.

■ MEMORIZAR

"Taint rejeita no nó, toleration aceita no pod. Ready entra no tráfego, Liveness ressuscita. cgroup v2 = systemd." Quatro frases garantem as questões de K8s da FGV 2023-2026.

Cluster: do Beowulf ao HPC

- **Server Farm**: conjunto de servidores que o cliente enxerga como um **único site lógico** — para requisições web massivas (não é IoT, virtualização, middleware ou P2P).
- **Beowulf**: hardware **comum e de baixo custo**, alto desempenho **escalável**, exige **rede** para comunicar os nós. As três assertivas → **F–V–F**.
- **Cluster HPC**: a **interconexão de alta velocidade** entre nós é essencial; nós têm múltiplas CPUs/GPUs, podem ser distribuídos geograficamente e escalam horizontalmente.
- **Alta disponibilidade**: cluster HA exige **redundância** e **tolerância a falhas** (hardware, software, energia). Pegadinha de vocabulário: **switchover** = troca manual; **failover** = substituição automática (e não "devolução do serviço à máquina de origem").

Nuvem: NIST, IaaS/PaaS/SaaS e nuvem nativa

NIST SP 800-145 — 5 características essenciais:

- ■ **On-demand self-service** — provisiona sem falar com o provedor
- ■ **Broad network access** — acesso por rede e plataformas heterogêneas
- ■ **Resource pooling** — recursos compartilhados entre consumidores, alocados dinamicamente
- ■ **Rapid elasticity** — expandir/reduzir quase instantâneo
- ■ **Measured service** — serviço medido/monitorado (pay-as-you-go)

■■ PEGADINHA

cobra **resource pooling** puro: "recursos físicos e virtuais compartilhados entre vários consumidores, atribuídos e retirados dinamicamente". A pegadinha mental: você vê "dinamicamente" e pensa em elasticidade — mas o enunciado fala de **compartilhamento entre consumidores**, não de expandir por demanda.

- **IaaS**: máquinas virtuais, armazenamento e redes **sob demanda**, pagando só pelo que usa, **sem hardware físico** — a escolha de startups que querem flexibilidade e controle. **PaaS** = plataforma para desenvolvedores. **SaaS** = aplicações prontas. Aplicativos executados no ambiente virtual → PaaS; eliminar aquisição de software → SaaS.
- **Elasticidade**: "capacidade de expandir ou reduzir dinamicamente seus recursos de acordo com a demanda".
- **Computação na nuvem**: acesso remoto a softwares, armazenamento e processamento via Internet.
- **Nuvem nativa**: responsivas, **escaláveis e tolerantes a falhas**, em nuvens públicas, privadas ou híbridas; modular e de baixo acoplamento (microserviços), não "Serverless-only".
- **DDoS**: vários servidores controlados inundam um alvo → **DDoS** (Distributed Denial of Service) — não adware, trojan, keylogger ou worm.

■ CONEXAO

O trio IaaS/PaaS/SaaS também é cobrado em **Governança de TI** e o NIST reaparece em **Segurança da Informação**. Mesma base teórica, três disciplinas do edital — economize uma leitura por prova.

Pegadinha final: rótulo da plataforma vs. conteúdo

Em provas de Infraestrutura, atenção ao vocabulário da assertiva: questões sobre **clusters de alta disponibilidade (HA)** e **balanceamento de carga** cobram tolerância a falhas e distribuição de requisições, enquanto menções a clusters **Hadoop/Spark** tratam de processamento distribuído de *Big Data*.

Resumo-final

> ■ RESUMO
>
> **Virtualização abstrai o físico do lógico. vMotion migra, DRS balanceia, HA reinicia após falha, FT espelha. Docker empacota contêiner; Kubernetes orquestra (pods, probes, taints, HPA, namespaces, GC). Cluster: Server Farm = um site lógico; Beowulf = barato e escalável, precisa de rede; HPC = interconexão rápida; HA = redundância (failover automático × switchover manual). Nuvem NIST: on-demand, broad access, pooling, elasticidade, measured service. IaaS = infraestrutura sob demanda; PaaS = plataforma; SaaS = software.** é a jóia da coroa — domine o resource pooling.

Infraestrutura T05 — Servidores de Aplicação (IIS e Apache)

> ■■ INFO
>
> **Servidores Web e de Aplicação (Apache HTTP Server, Microsoft IIS e Apache Tomcat):** arquitetura de diretórios e configuração (`httpd.conf`, `server.xml`), diretivas essenciais (`DocumentRoot`, `ServerRoot`, `DirectoryIndex`, `ServerTokens`), módulos de autenticação e comparação entre Apache e IIS.

Do que se trata?

Um restaurante. O **Apache** e o **IIS** são os garçons que levam os pratos (páginas) ao cliente. O **Apache Tomcat** é a cozinha especializada em Java — serve servlets. Para a FGV, o que importa é onde o garçom **guarda as configurações**, **que arquivo de diário ele anota** e **como ele se apresenta** ao cliente.

Os dois grandes: Apache × IIS

- **Apache**: **código aberto**, mundo **Linux/Unix**, flexível via módulos e arquivos de configuração.
- **IIS** (Internet Information Services): **proprietário, Microsoft**, mundo **Windows**, casa do ASP.NET/.NET.

> ■■ PEGADINHA
>
> As assertivas falsas clássicas: "Apache suporta nativamente todas as tecnologias Microsoft (ASP.NET)" → **falso**; "IIS não suporta PHP" → **falso** (suporta com configuração); "ambos são gratuitos e de código aberto" → **falso** (IIS é proprietário); "IIS é servidor de e-mail/banco" → **falso** (é servidor web).

Tomcat: o mestre das configurações

O arquivo **server.xml** (na pasta `conf` do Tomcat) é o mestre de configuração. A primeiríssima linha de configuração define o desligamento seguro:

```
<Server port="8005" shutdown="SHUTDOWN">
```

- **port** → define a **porta TCP/IP** que aguarda o comando de desligamento.
- **shutdown** → é a **string** do comando (SHUTDOWN), não a porta.
- **address** → endereço de escuta; **portOffset** → deslocamento sobre uma porta base. Nenhum dos dois nomeia a porta.
- O arquivo é **conf/server.xml** — jamais context.xml, web.xml ou config.xml.

■■ PEGADINHA

troca port por shutdown, address ou portOffset e server.xml por context.xml/web.xml. Regra de ouro: **port define a porta, shutdown define a senha de desligamento**. Separe os dois na hora da prova.

- **Classes Valve**: no Tomcat 10, são **pré-processadores das requisições do contêiner Catalina** — interceptam requisição/resposta antes do servlet (autenticação, redirecionamento, compressão). Não são realm (bancos de usuário), tags JSP nem hot deployment.

Logs clássicos do Apache

Log	Registra
	access.log
	agent.log
	referer.log
	error.log

■ DICA

agent → user-AGENT → navegador. A palavra-chave aparece literalmente no cabeçalho HTTP que identifica o browser. "custom.log", "format.log" e "rewrite.log" não são logs padrão da instalação.

Diretivas de configuração

- **DocumentRoot**: a pasta raiz de onde o Apache busca os arquivos da URL. Com `/opt/www`, a URL `http://www.cvm.com/9.10/index.html` procura `/opt/www/9.10/index.html`. Não é DirectoryIndex, ServerRoot, MaxSpareServers nem UserDir.
- **Host virtual**: as três diretivas válidas são **ServerAdmin** (e-mail), **DocumentRoot** (diretório) e **ServerName** (domínio). `EmailAdmin`, `DirectoryRoot` e `DomainName` **não existem** — a banca inventa diretivas para você eliminar.
- **mod_rewrite**: reescreve URLs em tempo real. Redirecionar `/bananaphone` → `/produtos/bananaphone` = regra 301 (`RewriteRule ^bananaphone$ /produtos/bananaphone [R=301,L]`). Não mexe em DNS, não atualiza o servidor, não usa página 404 nem VirtualHost.
- **Options -Indexes**: desativa a **listagem de diretórios** quando não há index.html. `+Indexes` faria o oposto.

- **SetEnv**: define **variável de ambiente** de forma incondicional (`SetEnv NOME "valor"`). Httpd é o programa; Worker é MPM; SetCore e ConfEnv não existem.
- **ServerTokens**: controla o **campo Server do cabeçalho HTTP**. `Prod` revela só "Apache"; `OS` revela produto+versão+SO (o que João queria esconder); `Full` revela tudo. ServerSignature controla o rodapé das páginas de erro — não o cabeçalho.

■ **MEMORIZAR**

"DocumentRoot manda onde buscar, ServerName identifica, ServerAdmin é o e-mail, ServerTokens esconde a versão, SetEnv define variável, -Indexes esconde a listagem." Decore pela função, não pelo nome.

Segurança: autenticação e ataques

- **Grupo "authentication type"**: **mod_auth_basic** (HTTP Basic, requer HTTPS) e **mod_auth_digest** (HTTP Digest, não manda senha em texto claro). Os demais (mod_auth_dbm, mod_auth_file, mod_authn_*, mod_authnz_ldap) são de armazenamento de credenciais/LDAP, não do tipo de autenticação.
- **Mitigação de DoS**: **threads via MPMs** (Worker/Event) = mais conexões simultâneas. Aumentar TimeOut piora (segura conexões); reduzir MaxRequestWorkers limita o legítimo; RequestReadTimeout só ajuda contra ataques lentos (Slowloris).
- **DDoS**: múltiplos servidores controlados inundando um alvo = **DDoS** (não adware, trojan, keylogger ou worm). De quebra: reduzir a informação exposta do servidor (ServerTokens/ServerSignature) dificulta o reconhecimento do atacante.
- **zcat**: lê arquivos **gzip sem descompactar** (economia de disco). `tar` empacota, `grep` busca texto, `touch` cria/atualiza timestamps, `expnd` não existe.

Ferramenta de teste: JMeter

- **Samplers** enviam requisições (cada um para um protocolo). — **Correto**
- **Thread Group**: usuários virtuais NÃO são iniciados simultaneamente por padrão — seguem o ramp-up; Timers adicionam atraso. — **Incorreto**
- **Assertions** validam a resposta (conteúdo, tempo, status). — **Correto**
- Gabarito: **I e III, apenas**.

Conexões com outras matérias

- **Redes**: User-Agent, campo Server e cabeçalhos HTTP vêm do protocolo HTTP (Tópico de Redes) — a FGV cobra a mesma teoria em duas disciplinas.
- **Portugues/Banco**: diretivas com nomes inventados são a pegadinha-padrão — se o nome não existe na doc oficial, a alternativa é falsa.
- **Infra T06**: Nginx/Apache como servidores de páginas HTML é o vizinho do T05 — as diretivas de Apache (DocumentRoot, ServerTokens) caem nos dois.

Resumo-final

■ **RESUMO**

> **Apache = open source/Linux; IIS = Microsoft/Windows. Tomcat vive no server.xml (port = porta de desligamento, shutdown = comando). Logs: access, agent (navegador), referer, error. Diretivas: DocumentRoot (raiz), ServerName/ServerAdmin (host virtual), SetEnv (variável), ServerTokens (cabeçalho), Options -Indexes (sem listagem), mod_rewrite (URLs). Auth: basic e digest. DoS: MPMs com threads. JMeter: Samplers enviam, Assertions validam.** Se a assertiva inventar diretiva (EmailAdmin, DirectoryRoot, LoadShare...), elimine na hora.

Infraestrutura — T06: Administração de Sistemas Operacionais (Linux e Windows)

> ■ **Visão Geral**
>
> A administração de servidores em ambiente corporativo exige domínio profundo do terminal GNU/Linux (gestão de processos, sinais, prioridades, usuários, compactação e serviços) e do prompt de comando do Microsoft Windows para automação de tarefas e suporte.

GNU/Linux: Gerenciamento de Processos, Prioridades e Sinais

No Linux, todo programa em execução é um processo com um identificador numérico único (**PID** — *Process ID*).

1.1 Prioridades de Execução: `nice` e `renice`

O escalonador de processos do Linux define a prioridade de CPU através de um valor chamado **niceness** (cortesia), que varia de **-20 a +19**:

- **Valor -20:** Prioridade máxima ("pouco cortês", consome o máximo de CPU). Apenas o superusuário (**root**) pode definir ou atribuir valores negativos!
- **Valor 0:** Prioridade padrão dos processos normais de usuário.
- **Valor +19:** Prioridade mínima ("muito cortês", cede CPU para todos os outros). Qualquer usuário comum pode aumentar a cortesia do seu próprio processo.

```
# Iniciar um processo novo com prioridade alta (-10) — exige root:
nice -n -10 backup_diario.sh

# Alterar a prioridade de um processo já em execução (PID 20284):
renice -n -5 -p 20284
```

> ■■ **Menor número = Maior prioridade!**
>
> No Linux, o número do `nice` funciona de forma invertida: quanto **menor** o valor numérico (-20), **maior** é a prioridade na fila da CPU. O valor positivo (+19) significa prioridade rasteira.

1.2 Controle de Processos e Sinais (`kill`)

O comando `kill` não serve apenas para "matar" processos; ele é um emissor de **sinais de controle** do kernel para o processo:

Sinal	Número	Nome Simbólico	Comportamento
	SIGTERM	**15**	Terminação Suave (Padrão)
	SIGKILL	**9**	Terminação Forçada e Imediata
	SIGHUP	**1**	Reiniciar / Recarregar

```
# Encerramento suave (envia SIGTERM 15):
kill 3175

# Encerramento forçado imediato (envia SIGKILL 9):
kill -9 3175
kill -SIGKILL 3175
```

GNU/Linux: Localização, Compactação e Usuários

2.1 Localização de Arquivos: `ls` vs `find`

- `ls`: Lista o conteúdo do diretório atual ou especificado.
- `find`: Ferramenta poderosa de busca recursiva baseada em critérios avançados:
  - `find /var/log -name "*.log"` (por nome).
  - `find /home -size +100M` (por tamanho, maior que 100 megabytes).
  - `find /tmp -mtime -7` (modificados nos últimos 7 dias).

2.2 Compactação e Criptografia: `tar` e `gpg`

- `tar` **(*Tape Archive*):** Empacota múltiplos arquivos em um único contêiner sem comprimir (extensão `.tar`). Para comprimir, combinam-se parâmetros:
  - `-c`: Criar arquivo (*create*).
  - `-x`: Extrair arquivo (*extract*).
  - `-v`: Modo detalhado (*verbose*).
  - `-f`: Especificar o nome do arquivo de destino (*file*).
  - `-z`: Comprimir usando o algoritmo **gzip** (extensão `.tar.gz`).
  - `-j`: Comprimir usando o algoritmo **bzip2** (extensão `.tar.bz2`).
- `gpg` **(*GNU Privacy Guard*):** Criptografa arquivos usando algoritmos simétricos ou assimétricos:

```
# Empacotar e criptografar um backup corporativo:
tar -czf /tmp/backup.tar.gz /home/usuario/
gpg -c /tmp/backup.tar.gz # Gera /tmp/backup.tar.gz.gpg
```

2.3 Gerenciamento de Usuários e Grupos

- `/etc/passwd`: Cadastro de contas de usuário (armazena login, UID, GID, diretório home e shell padrão).

- `/etc/shadow`: Armazena os hashes das senhas criptografadas e políticas de expiração (acesso restrito ao root).
- `/etc/group`: Cadastro de grupos do sistema e seus membros.
- Comandos: `useradd` (cria usuário no baixo nível), `passwd` (altera senha), `groupadd` (cria grupo) e `usermod -aG` (adiciona usuário a grupo complementar).

GNU/Linux: Servidor de Impressão CUPS

O **CUPS** (*Common UNIX Printing System*) é o subsistema de impressão padrão do Linux:

- Opera sobre o protocolo **IPP** (*Internet Printing Protocol*), utilizando a porta padrão **631 (TCP/UDP)**.
- O arquivo principal de configuração é o `/etc/cups/cupsd.conf`.
- `cupsctl`**:** Utilitário oficial de linha de comando para consultar e alterar diretivas do servidor CUPS (ex.: ativar compartilhamento de impressoras na rede com `cupsctl --share-printers`).

Microsoft Windows: Prompt de Comando (CMD) Essencial

Na administração de servidores Windows, o Prompt de Comando (CMD) fundamenta scripts em lote (`.bat` / `.cmd`):

Comando	Significado / Origem	Função	Pegadinha de Prova
	`md` **ou** `mkdir`	*Make Directory*	Cria um novo diretório
	`cd` **ou** `chdir`	*Change Directory*	Entra ou navega entre pastas
	`rd` **ou** `rmdir`	*Remove Directory*	Remove pastas
	`del` **ou** `erase`	*Delete / Erase*	Apaga **arquivos**, nunca pastas!
	`rem`	*Remark*	**Linha de comentário em scripts**

■ A armadilha do `rem` no Windows

As bancas costumam colocar o comando `rem` fingindo que ele significa "Remove" ou "Remover arquivo". Isso é pegadinha! Quem remove pasta é `rd` e quem remove arquivo é `del`. O `rem` vem de *Remark* (comentário).

Infraestrutura T07 — Armazenamento Orientado a Objetos (S3)

■■ INFO

Armazenamento Orientado a Objetos (*Object Storage* / Amazon S3 / MinIO): estrutura plana em *buckets* (sem hierarquia real de pastas), imutabilidade de objetos (chave única + dados + metadados ricos), acesso via API REST/HTTP e comparação direta entre armazenamento em Bloco (SAN/EBS), Arquivo (NAS/EFS) e Objeto (S3).

Do que se trata?

Imagina uma garagem de guardar-móveis. Você não organiza em pastas, não coloca num "HD": você entrega cada caixa com uma **etiqueta única** e o depósito guarda. É isso que o **object storage** faz — cada dado é um **objeto**: conteúdo + metadados + um **identificador único** (chave/UUID). Não existe hierarquia de pastas: é um mar de etiquetas.

O exemplo mais famoso é o **Amazon S3** (Simple Storage Service). E os "armazenamentos de arquivo" do dia a dia — **Dropbox, Google Drive, OneDrive** — vivem no mesmo guarda-roupas da nuvem, cada um com seu provedor.

■ **RESUMO**

Objetos = "conteúdo de internet" + backup. Fotos, vídeos, arquivos estáticos, backups: dados **não estruturados**, guardados como unidades independentes com ID único, em **buckets**. Perfeito para escala gigante e baixo custo — em detrimento de performance de recuperação e de estruturas hierárquicas.

Imutabilidade (WORM) — a pegadinha do "1 KB": o object storage é, por natureza, **WORM** (*Write Once, Read Many*) — você grava e a leitura passa a ser o forte. Consequência que a banca cobra: num objeto de **10 GB**, se você precisar mudar **1 KB** dele, não existe "editar o pedacinho" — você **sobrescreve o objeto inteiro** (reenvia os 10 GB). E a asserção clássica: a III ("elimina diretórios e pastas **e por isso é boa para dados dinâmicos que mudam o tempo todo**") é **falsa no final** — dispensar hierarquia é verdade; servir a workloads altamente mutáveis, não (o gabarito sai I e II apenas).

■ **ALERTA**

Duas verdades que andam juntas: (1) objeto = **dados + metadados + ID único**, acessível por **API**; (2) pode morar em **data center, nuvem ou híbrido**, acessível de qualquer lugar. Se a assertiva fechar com "indicado para dados que mudam constantemente", ela é **falsa** — para mutável, a banca prefere **bloco/arquivo**.

Os três tipos: onde a banca embaralha

Tipo	Analogia	Organização	Uso típico	Exemplo
	Objetos	depósito de caixas etiquetadas	objetos com ID único, sem pastas	fotos, vídeos, backups
	Arquivos	servidor de arquivos da empresa	**hierárquica** (pastas)	compartilhar em rede
	Blocos	HD de altíssimo desempenho	blocos de tamanho fixo	bancos de dados

■■ **PEGADINHA**

A pegadinha-rei: "estruturado, não estruturado e semiestruturado" são tipos de **DADOS**, não de **ARMAZENAMENTO** (é a descrição do conteúdo, não do contêiner). E "diretórios", "links", "hashes" e "cadeias de blocos" também não são tipos de armazenamento em nuvem. A correta é sempre **objetos, arquivos e blocos**.

O caso de negócio: quando escolher objeto vs. bloco

A questão (Banco do Brasil) monta o cenário clássico: Marta quer **backup de ativos de multimídia** (áudio, vídeo, imagem). Prioridades: **escalabilidade ilimitada + menor custo**, aceitando não recuperar rápido e não ter estrutura de pastas. O object storage casa perfeito: altamente escalável, ótimo para não estruturados, econômico. **Blocos** são ideais para desempenho consistente (banco de dados) — mas caros em escala de mídia.

> **■ ALERTA**
>
> Antes de marcar "blocos" por impulso (todo mundo pensa em performance), leia o enunciado: se ele fala em **backup, grande volume, custo baixo, dados não estruturados, sem hierarquia**, a banca está descrevendo **objetos**.

Nuvem: o par de cada provedor

Serviço	Provedor	Características
	S3	Amazon (AWS)
	Dropbox	Dropbox
	Google Drive	Google
	OneDrive	Microsoft

A FGV exige saber **quem é de quem**.: serviços usáveis "a qualquer momento e de qualquer lugar na web" → **s3 e dropbox**.: "nomes de serviços de armazenamento em nuvem" → **Google Drive e OneDrive**.: o serviço da **Microsoft** → **OneDrive** (nunca Dropbox, Drive, AWS ou Bing — Bing é buscador).

> **■ MEMORIZAR**
>
> **Microsoft → OneDrive; Google → Drive; Amazon → S3; Dropbox → Dropbox.** Se vier sigla de modelo (SaaS, IaaS, PaaS), "b64", "iass" ou plugins de navegador (Edge, Chrome), é **distrator** — não são serviços de armazenamento.

Peculiaridades funcionais da FGV

- **Retenção** (OneDrive): preservar arquivos de contas excluídas por um período → configurar **retenção** (não "lixeira", "restauração", "sincronização" ou "pré-provisionamento").
- **Ícone de nuvem azul** (OneDrive/SharePoint no Explorador do Windows): arquivo **disponível apenas online** (não baixado para o disco). Sincronizando = setas circulares; bloqueado = cadeado; compartilhado = "pessoas".

Buckets e o MinIO na prática

Bucket é o "contêiner" do object storage. O **MinIO** é um servidor de armazenamento compatível com S3, e o cliente de linha de comando é o **mc**.

```
mc mb tjrnS3/site.estatico
```

- **mb** = *make bucket* → cria o bucket.
- Sintaxe: `mb [ALIAS]/[NOME_DO_BUCKET]`.
- **od** não existe para isso; **du** mostra uso de armazenamento, não cria bucket.

■ **DICA**

mb = make bucket. Na dúvida, decodifique o comando: `mb` faz, `od` e `du` não fazem. E o alias vem **antes** do nome do bucket, separado por barra.

Vantagens e desvantagens do armazenamento em nuvem

A FGV cobra o julgamento de afirmativas sobre cloud storage:

- **Vantagens**: acesso de qualquer lugar/dispositivo, compartilhamento (****: compartilhar arquivos e fotos com colegas), backup automático, colaboração, **continuidade de negócios** (dados seguros mesmo com desastre físico local), criptografia e varredura proativa de ameaças.
- **Desvantagens**: **dependência de internet** (qualidade do acesso ligada à conexão), risco de **violação de dados** sempre existente (nenhum sistema é 100% seguro), possível lentidão em redes públicas/compartilhadas.

■■ **PEGADINHA**

A pegadinha da FGV: a alternativa que "elimina" uma afirmativa por considerar segurança perfeita ou performance garantida. As suas três afirmativas clássicas são **todas verdadeiras** e equilibradas — quando o enunciado não exagera com "sempre/nunca/apenas", o gabarito costuma ser o mais abrangente. E "cloud storage" = **servidores remotos acessíveis pela internet**, jamais disco local, CD/DVD ou pen drive.

Conexões com outras matérias

- **Banco de Dados**: banco de dados relacional é o cliente clássico de **blocos** (T03 SAN/RAID, EBS); objeto fica com o não estruturado (data lake, Big Data).
- **Redes**: S3/MinIO rodam sobre HTTP/HTTPS — mesmo chão da web do T05/T06.
- **Governança**: retenção de arquivos ecoa política de retenção de dados da LGPD — segurança da informação negocia com a TI.
- **T03**: o armazenamento em nuvem herda a lição do T03 — o **nível de acesso** (objeto/arquivo/bloco) manda na classificação, e a banca mistura rótulos de propósito.

Resumo-final

■ **RESUMO**

Object storage = grandes volumes de dados não estruturados, ID único, buckets, baixo custo, escalável (S3, MinIO, Azure Blob). Arquivos = hierarquia de pastas (Drive, Dropbox, EFS). Blocos = desempenho para banco de dados (EBS). OneDrive é da Microsoft, Drive é do Google, S3 é da Amazon. Comando de bucket no MinIO: `mc mb alias/bucket`. Vantagem da nuvem = compartilhar + continuidade; desvantagem = dependência de internet e risco residual de violação.
